

Machine Learning for teens

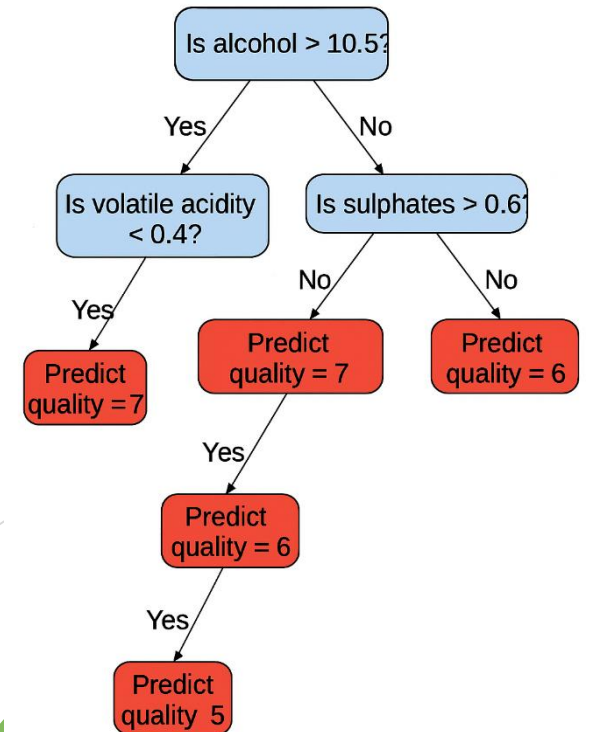
By Hamid Sadeghi

Table of Contents

- ▶ Decision Tree
- ▶ Types of fitting
 - ▶ Underfitting
 - ▶ Appropriate Fit
 - ▶ Overfitting
- ▶ Overcome overfitting
 - ▶ Feature selection
 - ▶ SelectKBest
 - ▶ Random Forest
 - ▶ Lasso
 - ▶ Model Constraints

Decision Tree

- ▶ Decision Tree .download data from [here](#)
- ▶ A **Decision Tree** is a supervised learning algorithm used for both **classification** and **regression**. It models decisions as a tree-like structure, where each node represents a test on a feature, each branch represents an outcome of the test, and each leaf node represents a final prediction.
- ▶ Let's say we're using just three features:
 - ▶ Alcohol
 - ▶ Volatile Acidity
 - ▶ Sulphate



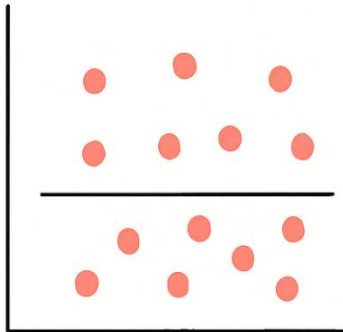
Types of fitting

- ▶ It performs well on training data but poorly on test data
- ▶ Types of fitting
 - ▶ ✓ **Underfitting:** The model is too simple – it misses the pattern entirely.
 - ▶ ✓ **Appropriate Fit:** The model captures the general trend without considering noise.
 - ▶ ✓ **Overfitting:** The model is too complex and captures everything in the training data, including noise.

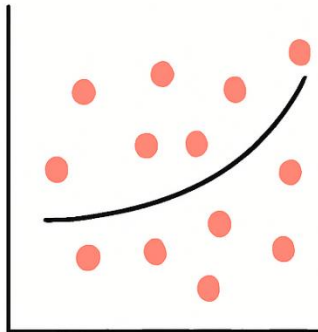
Fit Type	Training Score	Test Score
Underfitting	Low	Low
Appropriate Fit	High	High
Overfitting	Very High	Low

Types of fitting

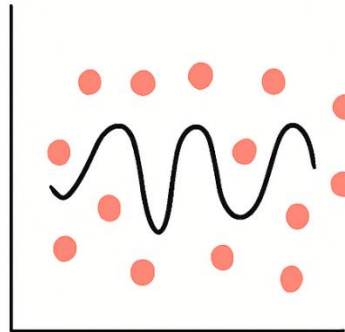
UNDERFITTING



GOOD FIT



OVERFITTING



Feature selection

- ▶ removing irrelevant features is one of the most direct and effective ways to reduce model complexity and improve generalization.
- ▶ Feature selection
 - ▶ SelectKBest
 - ▶ is a feature selection method from **scikit-learn** that helps you automatically pick the top **K most relevant features** based on a scoring function.
 - ▶ Random Forest
 - ▶ is a machine learning method that builds many decision trees. While training, it learns which features help the model make better predictions. It gives each feature a score based on how useful it was.
 - ▶ Lasso (L1 regularization)
 - ▶ is a smart version of linear regression. It looks at all your features and figures out which ones actually help predict the target

Random forest

- ▶ Random Forest is made up of many decision trees
- ▶ Decision Trees and Random Forests can choose the right features by themselves. You don't always need to do feature selection. What matters more is keeping the model simple so it doesn't over fit.
 - ▶ `dt_model = DecisionTreeClassifier(`
 - ▶ `max_depth=5,` `# Limit how deep the tree can grow`
 - ▶ `min_samples_split=10,` `# Require more samples to split a node`
 - ▶ `min_samples_leaf=5,` `# Require more samples in leaf nodes`
 - ▶ `random_state=42`
 - ▶ `)`

Ways to Overcome Overfitting

- ▶ **Feature Selection**
 - ▶ Simplify the model by removing irrelevant or noisy features.
- ▶ **Regularization**
 - ▶ Regularization works for linear models.
- ▶ **Cross-Validation**
 - ▶ KFold, StratifiedKFold, GridSearchCV
- ▶ **Model Constraints (for Trees)**
 - ▶ Like our code
- ▶ **Dimensionality Reduction**
 - ▶ Reduce feature space like PCA

Ways to Overcome Overfitting

- ▶ **Dropout**
 - ▶ **For neural networks**
- ▶ **Data Augmentation (for images/text)**
 - ▶ Increase training diversity.
 - ▶ Flip, rotate, crop images
 - ▶ Synonym replacement for text
- ▶ **Ensemble Methods**
 - ▶ Combine models to reduce variance.
 - ▶ Bagging (Random Forest), Boosting (XGBoost), Stacking

Assignment

- ▶ Upload [data](#)
- ▶ Do the taring
- ▶ Check if you face overfitting
- ▶ If you faced overfitting , try to overcome it